

1. Functional Testing
 2. Boundary Value Testing
 3. Input Boundary Value Testing (BVT)
 4. Robustness Testing
 5. Pros and Cons of Boundary Value Testing (BVT)
-

Functional Testing

A program is treated as a mathematical function (F) that maps an **Input Domain** to an **Output Range**.

- **Equation:**
Output = F(Input)
- **Input Domain:** All possible input variables (x1,x2,..xn).
- **Output Range:** The expected results based on the mapping.

Boundary Value Testing (BVT)

Definition: A black-box testing technique that focuses on identifying errors at the **boundaries** (edges) of input ranges rather than in the center.

Two Core Considerations

- **Invalid Values:** Should we test inputs outside the valid range? (Leads to **Robustness**).
- **Single Fault Assumption:** Do we assume only one variable fails at a time? (Leads to **Worst-Case**).

Pros and Cons of Boundary Value Testing (BVT)

1. Advantages

- **Tool Support:** Plenty of commercial tools are available to help.
- **Efficiency:** Very easy to execute manually and simple to automate.
- **Specific Fit:** Best for calculation-intensive apps with physical units

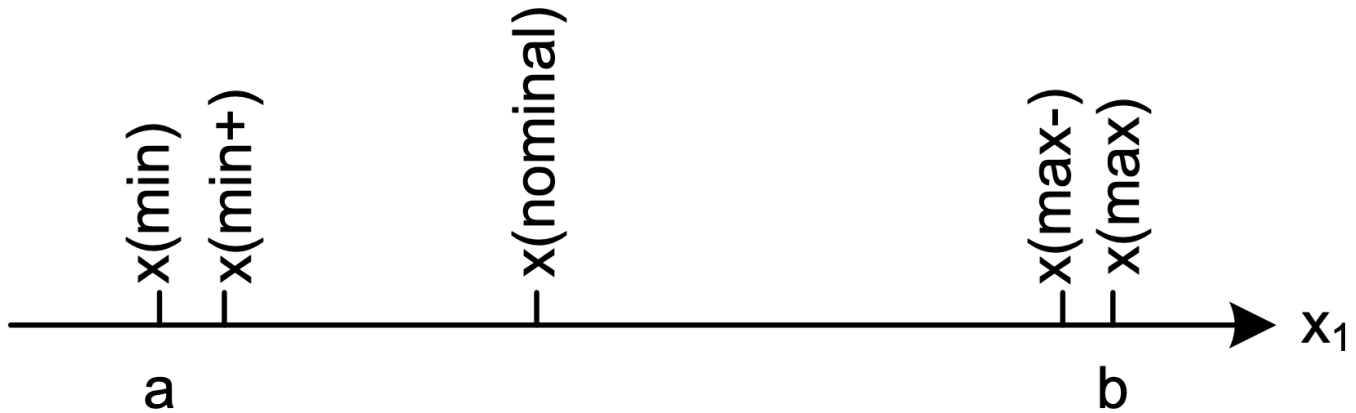
2. Disadvantages

- **Coverage Flaws:** High potential for both **gaps** and **redundancies**
- **Blind Spots:** Because it is specification-based, identifying these gaps or redundancies is nearly impossible.

- **Poor Scalability:** Does not scale well as the number of variables increases.
- **Output Manual:** Tools only generate the **inputs**, the tester must still manually define the expected outputs.

Input Boundary Value Testing (BVT)

Definition: A testing technique where test cases are designed using the boundary values of an input variable. It focuses on the limits where the system behavior is likely to change.

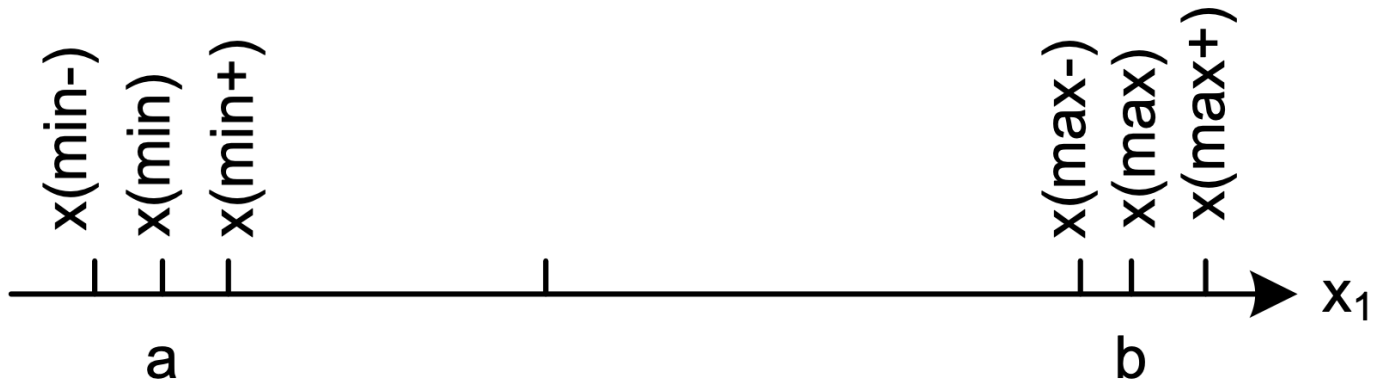


- Test values for variable x , where
 - $a \leq x_1 \leq b$, and
 - $x(\min)$, $x(\min+)$, ... $x(\max)$ are the names from the T tool.

Robustness Testing

Definition: An extension of Boundary Value Testing (BVT) that includes **invalid values** to verify how the system handles out-of-range inputs.

Robustness Testing



Advantages:

It verifies error handling, uncovers unintended hidden logic, and provides a baseline for exploratory testing.

Challenges:

Expected results for invalid inputs are often undefined, and strong typing in compilers can make these tests redundant.